

太陽観測データ自動整理システム 開発企画書

観測事後整理システム構想 (after_kansoku) に基づく要件定義と基本設計 ver0.2

作成者: 島野淳

作成日: 2026年6月4日

バージョン: ver0.2 (オリジナル構想・生データ構成統合版)

1. プロジェクト概要

本プロジェクトは、太陽観測において取得される膨大な画像データと高頻度な気象データをタイムスタンプを軸に正確に紐づけ、観測後のデータ整理・解析を自動化するシステム「after_kansoku」の構築を目的とします。

これまでの検討に基づき、毎秒撮影される太陽のTIFF画像、AS!studioが出力するメタデータ、自作画像解析プログラム (MIN2, seeing_allframe)、および新たに要件に加わった太陽の東西検出機能 (west_solar.py) による結果を、128Hzの高頻度でサンプリングされた気象データ (verniers テーブル) と統合管理するデータ基盤を確立します。

2. システムの運用形態と基本フロー

本システムは、複雑な環境構築やSQL操作に不慣れなエンドユーザーでも安全かつ容易にデータ操作・閲覧が行えるよう、実運用に即した「Foolproof (誤操作防止)」な設計を採用します。

【基本使用手順】

1. **データ観測:** 観測時に、独自作成の気象条件観測システム (get_environment.py) により128Hzで環境データを一時的なDBへ記録・蓄積。
2. **設定ファイル (setting.txt) の準備:** 保存先 (savepath) や画像パス (picpath) 等のオプション環境を外部定義。
3. **システムの実行:** 配布された after_kansoku.exe をワンクリックで実行。
4. **解析の自動実行:** 内部で MIN2、seeing_allframe、および太陽の東西検出 (west_solar.py) を自動で呼び出し処理。
5. **データベース格納:** 指定した保存先フォルダに一元化されたSQLite形式のデータベース (.db) が自動保存・追記される。
6. **Excel複製変換 (Foolproof設計):** DB破損を防ぐため、「DB → Excel」への一方通行の処理 (db_to_xlsx) を実行し、閲覧用ファイルを生成。

【生データの構成】

システムが処理対象とする、外部ストレージ内の観測データおよびディレクトリ構造の想定は以下の通りです。

```
E:.\
└─observed_result
  │└─environ_yy-mm-dd-tt.txt      # 気象データ（テキスト形式）
  │└─weather_results.db          # 気象データ（DB形式）
  └─pictures                      # 太陽画像格納フォルダ
    │└─001
    │  │└─00001.tiff             # 観測写真本体
    │  │└─00001.txt             # camera_stat（撮影時のメタデータ）
    │  │└─00002.tiff
    │  └─00002.txt
    │└─002
    │  │└─00001.tiff
    │  └─00001.txt
```

3. 技術スタックおよび主要ライブラリ

開発言語には、強力なデータ解析エコシステムを持つPythonを基本とし、画像処理、センサー通信、データ解析を網羅する標準環境を採用します。すべてのデータハンドリングをSQL文の直接記述なしで完結させます。

項目	選定技術 / ライブラリ	役割・選定の理由
開発言語	Python	画像処理、センサー通信、データ解析を網羅する標準環境。
データベース	SQLite (sqlite3)	サーバーレスで動作するローカルファイル駆動型。SQL形式での確実なデータ保存。
データ処理	pandas	pandas.to_sql を用いた一括保存（バルクインサート）により、PCへの負荷を大幅に軽減。
Excel複製変換	openpyxl 等	エンドユーザー向けの閲覧用 .xlsx 複製ファイルを安全に出力（Foolproof設計）。
ハードウェア通信	godirect (Vernier公式)	Vernierセンサーからの高頻度（128Hz）データをダイレクトに取得。

項目	選定技術 / ライブラリ	役割・選定の理由
カメラ制御・画像処理	zwoasi, OpenCV (opencv-python), ASICamera2.dll	ASICameraからのリアルタイム画像取得（SDK/DLLを介した直接制御）、および東西検出プログラムにおけるリアルタイム画像処理・解析。
配布パッケージ	Nuitka	最適化モードを用い、Python非環境下でもより高速かつ効率的に動作するスタンドアロン実行ファイル（after_kansoku.exe）化。

4. ディレクトリ構成とコード構造

システム全体を「基盤・検証（basement）」「観測・通信（observation）」「メイン解析・処理（main）」の3つのスコープに分離し、高い保守性を担保します。

```

E:.
├─basement                                # 基盤テスト・デモ領域
│   ├── pandas_sample.py                 # Pandasを用いたDB操作デモ
│   ├── sql_sample.py                   # SQLを用いた初歩的データベース操作デモ
│   ├── testDB.db                       # デモスクリプトから自動生成されるテストDB
│   ├── 実装したいこと.md               # UI/UXや機能拡張に関するメモ
│   └── 観測事後整理システム構想ver0.1.md
├─observation                             # 観測・センサー・個別物理処理領域
│   ├── get_environment.py              # 気象観測ソフト（一時DBへ128Hzデータを蓄積）
│   └── west_solar.py                   # 太陽の東西検出ソフト
└─main                                    # メイン処理・司令塔領域
    ├── edit_database.py                # 【システムメイン】全体の司令塔スクリプト
    ├── picture_analysis.py             # 画像解析モジュール（MIN2, seeing_allframeを内包）
    └── setting.txt                     # 外部設定ファイル（savepath, picpath, オプションを定義）

```

各フォルダおよびスクリプトの役割定義

- **main / edit_database.py:** 本システムのコア。setting.txt からパス情報を取得し、対象の画像群を自動走査。picture_analysis.py を呼び出して結果を抽出し、気象データを統合して最終的なSQLite DBへ書き込み、さらにExcelへとエクスポートする役割を一手に担います。
- **main / picture_analysis.py:** ASISudioが書き出す「撮影情報.txt」の展開（パース）を行い、さらに既存の太陽画像解析（MIN2, seeing_allframe）を関数としてラッピング（モジュール化）したものです。for文の外側で一度だけインポートし使い回すことで、処理スピードの低下を完全に防ぎます。

- **observation / west_solar.py**: 撮影画像または観測データから太陽の東西方向を検出する独自ソフトです。事後処理（after）の段階でピクセルを汚さない形で方向情報を算出し、データベースへと記録します。
- **observation / get_environment.py**: 128Hzサンプリングという高頻度データをPCに負荷をかけずに受け取るため、1秒間（128個分）の配列データを確保。pd.date_range(..., freq='7.8125ms') を用いてズレのない精密なミリ秒タイムスタンプを自動生成し、一時DBへ一括保存（バルクインサート）します。
- **basement /**: 開発および検証用のデモ領域。PandasやSQLの初歩的な挙動確認スクリプト、テスト用DB、UI/UXの改善案や機能拡張のメモをまとめた「実装したいこと.md」などを格納します。

5. データベース構造（データモデル設計）

画像ファイル本体は外部ストレージ（HDD/NASなど）の構造化されたフォルダ（例: pictures/001/）に配置し、DB内にはその「ファイルパス」と「解析数値」のみを格納する王道のハイブリッド構成をとることで、パフォーマンスの最大化と容量の節約を実現します。

① photos テーブル（写真データ・解析結果準拠 / 1秒に1行のペース）

観測写真に紐づくメタデータおよび画像解析数値を格納します。

カラム名	データ型	説明 / 役割
shot_time	TIMESTAMP (TEXT)	【主キー】ASlstudioのテキストから展開した撮影時刻（秒精度）。
image_path	TEXT	TIFF写真の物理保存先パス（setting.txt の picpath 基準）。
sun_radius	REAL	【解析値】MIN2 等で自動検出した太陽の半径。
center_x	REAL	【解析値】太陽の中心位置（X座標）。
center_y	REAL	【解析値】太陽の中心位置（Y座標）。
img_seeing	REAL	【解析値】seeing_allframe から算出した大気揺らぎ数値。
solar_direction	TEXT	【解析値】west_solar.py で算出された太陽の東西方向。

② verniers テーブル（気象条件データ / 1秒に128行のペース）

Vernierセンサーから取得した、ミリ秒単位の高精度な気象条件データを保管します。

カラム名	データ型	説明 / 役割
sample_time	TIMESTAMP (TEXT)	【主キー】ミリ秒精度の観測時刻（pd.date_range で高精度生成）。
wind_speed	REAL	環境センサーより取得された風速。
humidity	REAL	環境センサーより取得された湿度。
pressure	REAL	環境センサーより取得された気圧。

【データ連携方法】

写真データと微細な気象条件データを組み合わせて分析する際は、写真の shot_time（秒精度）をキーとし、気象側の sample_time（ミリ秒精度）に対してSQLの BETWEEN 検索、あるいはPandasによる範囲条件抽出（例: shot_time から1秒間分）をかけることで、「1対多（1対128）」の関係を正確に紐付けます。

6. 開発タスクと実装方針

本企画書（ver0.2）に基づき、以下の5つの明確なタスクに沿って実用最低ラインをクリアするシステムを構築します。

1. 画像解析ソフトの統合・モジュール化: 作成済みの MIN2、seeing_allframe を picture_analysis.py 内に関数として内包し、Pandasの DataFrame 形式を介してSQL（SQLite）へ高速書き出しする機構を実装。
2. camera_stat (AS!studio) の展開ソフト作成: 00001.txt などのテキスト構造から正規表現等を用いて安全に撮影時刻、露出情報をパース。
3. vernier データ収集ソフトの高度化: 観測機器や観測時間をミリ秒単位まで高精度に記録し、かつバルクインサートで安全に保存する get_environment.py の確立。
4. db_to_xlsx 複製変換ソフトの実装: pd.read_sql_query で取得した全データを openpyxl を用いてExcelファイルへ複製する処理。エンドユーザーがExcelを誤って変更しても、元データ（.db）が壊れない一方通行（Foolproof）設計の徹底。
5. 太陽の東西検出ソフトの作成・結合: west_solar.py のロジックを確定させ、司令塔である edit_database.py からスムーズに解析数値を追記・統合できるようにする。

7. 将来の展望（Python 3.13への移行によるメリット）

現在は安定性を考慮して既存環境をベースとしますが、将来的に開発環境をPython 3.13にアップグレードすることで、以下の恩恵を受けられ、運用・保守の快適性が劇的に向上します。

- **pathlib.walk() の導入:** `observed_result/pictures/001/...` のように多階層に深く分かれた大量の写真フォルダ・画像・テキスト群を、OSに依存しない直感的なコードだけで一括走査可能。
- **自己診断型エラーメッセージの強化:** モジュール名の名前衝突（例: `re.py` や `pathlib.py` などの誤命名）やタイプが起きた際、原因をピンポイントで提案してくれるため、現場でのデバッグ時間が大幅短縮。
- **f-stringsの制限撤廃:** フォーマット文字列内での改行やクォーテーションの自由なネストが可能になり、複雑なSQLクエリ生成やログ出力コードの可読性が大幅向上。